



IIC2343 - Arquitectura de Computadores (I/2026)

Actividad de programación

Sección 2 - Pauta de evaluación

Pregunta 1: Explique el código (3 ptos.)

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_n_m`:

```
.data
n:      .word 2
m:      .word 5
res:    .word 0

.text
main:
    la t0, n
    la t1, m
    la t2, res
    lw a0, 0(t0)
    lw a1, 0(t1)
    addi sp, sp, -8
    sw ra, 0(sp)
    sw t2, 4(sp)
    jal ra, func_n_m
    lw ra, 0(sp)
    lw t2, 4(sp)
    addi sp, sp, 8
    sw a0, 0(t2)
    addi a7, zero, 10
    ecall

func_n_m:
    addi sp, sp, -8
    sw ra, 0(sp)
    sw a0, 4(sp)
    add t2, zero, zero
    beq a0, t2, base_end_1
    beq a1, t2, base_end_2
    addi a1, a1, -1
    jal ra, func_n_m
    lw t0, 4(sp)
    mul a0, t0, a0
    jal zero, end
base_end_1:
    add a0, zero, zero
    jal zero, end
base_end_2:
    addi a0, zero, 1
end:
    lw ra, 0(sp)
    addi sp, sp, 8
    jalr zero, 0(ra)
```

Este fragmento representa el cómputo de una función $f(n, m)$. A partir de este:

1. (1.5 ptos.) Indique, con argumentos y en términos de n y m , lo que retorna la función $f(n, m)$. Por ejemplo, $f(n, m) = n + m$. Se otorgan **0.75 ptos.** por la correctitud de la descripción del retorno y **0.75 ptos.** por justificación.

Solución: La función computa recursivamente la potencia de n a la m , *i.e.* $f(n, m) = n^m$. Esto se evidencia en el hecho de que el valor de retorno, almacenado en el registro **a0**, es igual a $n \times f(n, m - 1)$, con caso base $f(n, 0) = 1$. De esta forma, se tiene que:

$$f(n, m) = n \times f(n, m - 1) = n \times n \times f(n, m - 2) = \dots = n \times n \times \dots \times n \times 1 = n^m$$

No es necesario que se haga un detalle completo del código, pero sí que se demuestre que se entiende el cómputo recursivo de la potencia de n a la m .

2. (1.5 ptos.) Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V. Se otorgan **0.75 ptos.** si indica de forma correcta si se respeta o no la convención y **0 ptos.** si su respuesta es incorrecta. Por otra parte, se otorgan **0.75 ptos.** por entregar una justificación válida respecto a su respuesta.

Solución: El fragmento anterior **sí respeta** la convención de llamadas de RISC-V. Los motivos son los siguientes:

- Se respalda **ra** antes de cada llamado.
- Se usan los registros **a0** y **a1** para los argumentos de **func_n_m**, y **a0** para su retorno.
- Se respalda el registro **a0** antes de cada llamado recursivo, considerando su sobreescritura; su uso posterior al llamado; y el hecho de ser *caller-saved*.
- Se respalda el registro **t2** **antes** del llamado de la subrutina, considerando su sobreescritura; su uso posterior al llamado; y el hecho de ser *caller-saved*.

Pregunta 2: Elabore el código: Bloque más pesado (3 ptos.)

Elabore, utilizando el Assembly RISC-V, un programa que a partir de un arreglo `arr` de largo `len`, encuentre el **bloque contiguo más pesado**. Se define el peso de un bloque como el valor del número multiplicado por la cantidad de veces consecutivas que se repite. El programa debe almacenar el peso máximo encontrado en la variable `res`; puede asumir que $len \geq 1$. Si hay dos bloques con el mismo peso, considere el primero que se haya encontrado.

A continuación, cuatro ejemplos:

```
arr = [1, 1, 1, 2, 2, 5]  len = 6 → res = 5 × 1 = 5
arr = [3, 3, 2, 2, 2, 2]  len = 6 → res = 2 × 4 = 8
arr = [4]                len = 1 → res = 4 × 1 = 4
arr = [1, 1, 3, 1, 1, 1, 1] len = 7 → res = 1 × 4 = 4
```

Puede utilizar el siguiente fragmento de código como base:

```
.data
arr: .word 1, 1, 1, 2, 2, 5
len: .word 6
res: .word -1
.text
# Su código aquí.
# Puede utilizar este label para terminar el programa
end_program:
    addi a7, zero, 10
    ecall
```

La asignación de puntaje se distribuirá de la siguiente forma:

- **1 pto.** por identificar los quiebres entre bloques e iterar correctamente contando las repeticiones contiguas. Se descuentan **0.5 ptos.** si existe como máximo un error de implementación y no se asigna puntaje si existe más de uno.
- **2 ptos.** por realizar correctamente la ponderación del peso de cada bloque en las transiciones y guardar exitosamente el mayor valor resultante. Se descuenta **1 pto.** si existe como máximo un error de implementación y no se asigna puntaje si existe más de uno.

IMPORTANTE: No es necesario que respete la convención en este ejercicio.

Tips

- Puede usar la instrucción `mul` para computar el producto que genera el peso de un bloque, pero recuerde que también lo puede calcular a través de sumas sucesivas.
- No es necesario que haga el cómputo a través de subrutinas. Puede hacerlo de manera iterativa con saltos condicionales.
- Para obtener la dirección del i -ésimo elemento de un arreglo en RISC-V, puede usar esta fórmula: `dir(arr[i]) = dir(arr) + 4 × i`

Solución: A continuación, se muestra una solución que realiza el siguiente procedimiento iterativo:

- Si el valor de `arr[i]` es igual al del bloque actual, `t3`, entonces se incrementa su peso total almacenado en `t4`.
- Si son distintos, se revisa si el peso actual es mayor al último peso calculado.
- Solo si es mayor, se actualiza el máximo peso encontrado. Luego, se reestablecen los valores para inicializar al nuevo bloque actual.
- Se incrementa el valor de `i` para la siguiente iteración. En caso de que se haya revisado el último elemento del arreglo, entonces se vuelve a revisar si es que el peso actual es mayor al máximo encontrado. Esto es importante para el caso borde donde `len == 1`.
- Se hace la actualización de peso final, si corresponde, y termina el programa actualizando la variable `res` con el máximo encontrado, almacenado en `t5`.

```
.data
arr: .word 1, 1, 1, 2, 2, 5
len: .word 6
res: .word -1
.text
init:
    lw s0, len           # s0 = Largo arr
    la s1, res           # s1 = Dir. memoria res
    la t0, arr           # t0 = Dir. memoria arr[i]
    addi t1, zero, 0     # t1 = i = 0
    lw t3, arr           # t3 = valor bloque actual
    addi t4, zero, 0     # t4 = peso bloque actual
    lw t5, res           # t5 = peso máximo actual

loop:
    lw t2, 0(t0)         # t2 = arr[i], bloque actual
    beq t2, t3, inc_weight # if t2 == t3 -> mismo bloque, aumenta peso
    ble t4, t5, next_block # if t4 <= t5 -> bloque actual no pesa más, sigue al siguiente
    add t5, zero, t4     # t5 = nuevo peso máximo

next_block:
    add t3, zero, t2     # t3 = nuevo valor bloque actual
    add t4, zero, t3     # t4 = peso inicial nuevo bloque
    jal zero, next_iter  # Siguiendo iteración

inc_weight:
    add t4, t4, t3       # t4 += peso bloque actual

next_iter:
    addi t1, t1, 1       # t1 += 1 -> i += 1
    bge t1, s0, end_loop # if i > len, salta a end_loop
    addi t0, t0, 4       # t0 += 4 -> dir. sgte. elemento
    jal zero, loop       # Reinicia loop

end_loop:
    ble t4, t5, end_program # if t4 <= t5 -> bloque actual no pesa más, termina el programa
    add t5, zero, t4     # t5 = nuevo peso máximo y termina

end_program:
    sw t5, 0(s1)         # res = peso máximo encontrado
    addi a7, zero, 10
    ecall
```

Aparte de la distribución de puntaje antes señalada, se realizan las siguientes consideraciones:

- Si no se llega a la solución correcta en ningún caso, o bien el programa se cae, se otorgarán **0.75 ptos.** si se evidencia a nivel de código un intento de algoritmo cercano a lo esperado.
- Si el programa falla **solo** en el caso donde `len == 1`, se descuentan solo **0.5 ptos.** del total.